

Aula 03 - Ramificações (Branches), Tags e Correção de Erros

Descrição: Exploração de como criar "branches" e "tags" para testar novas ideias de forma segura, sem afetar o projeto principal. Também será abordado como configurar o sistema para ignorar arquivos indesejados e como corrigir erros comuns durante o desenvolvimento.

Objetivo da Aula: Capacitar os alunos a criarem ambientes de testes isolados (branches), blindar o repositório contra arquivos desnecessários (.gitignore), marcar marcos históricos no projeto (tags) e perder o medo de errar, aprendendo a reverter alterações de forma segura.

Dando sequência à nossa jornada de aprendizado, a Aula 03 é o momento em que vocês ganham maturidade no uso do Git. Vocês vão deixar de ver o Git apenas como um "salvador de arquivos" e passarão a tratá-lo como um **ambiente seguro para experimentação**.

1. O Conceito: A Linha do Tempo Paralela e o Escudo do Repositório

- **Branches (Ramificações):** Uma branch é como uma realidade alternativa do projeto. Você pode criar uma nova ramificação para testar um código arriscado ou desenvolver uma nova tela. Se der errado, basta apagar a branch e a versão principal (estável) continuará intacta.
- **Tags (Etiquetas):** São "post-its" fixados em commits específicos para marcar versões importantes do software (ex: v1.0.0-lançamento). Diferente das branches, as tags não mudam; elas servem como marcos históricos.
- **O .gitignore:** Nem tudo no computador deve ir para o GitHub. Arquivos de configuração pessoal, senhas, fotos pesadas ou pastas de dependências devem ser ignorados. O .gitignore é o escudo que diz ao Git o que ele deve fingir que não está vendo.

2. Passo a Passo Prático

Passo 1: O Escudo contra Arquivos Indesejados (.gitignore)

Arquivos sensíveis não devem ser rastreados pelo Git. Para configurar isso:

1. Criem um arquivo na raiz do repositório chamado .gitignore (atenção ao ponto no início do nome).
Nota para Windows: O comando touch .gitignore funciona no terminal do Git Bash (que emula Unix). No Prompt de Comando padrão (CMD), ele poderá falhar. Outra alternativa é criar o arquivo diretamente pelo VS Code.

2. Dentro dele, adicione as regras em linhas separadas:

```
# Arquivos de configurações e rascunhos pessoais
```

```
config_secreta.txt
```

```
notas_secretas.txt
```

```
# Ignorar qualquer arquivo que termine com a extensão .log
```

```
*.log
```

3. Crie um arquivo chamado suporte.log na pasta. Rode git status e verifique que o Git simplesmente não lista esse arquivo, pois ele está devidamente blindado.

Passo 2: Criando e Navegando em Branches (git branch e git checkout)

Para criar uma nova ramificação e entrar nela:

1. Veja as branches existentes (a atual estará com um asterisco):
`git branch`
2. Crie uma nova branch chamada nova-ideia:
`git branch nova-ideia`
3. Entre na nova branch:
`git checkout nova-ideia`

Dica de produtividade: O comando rápido `git checkout -b nova-ideia` faz as duas ações de uma só vez.

Passo 3: Marcando Versões com Tags (git tag)

Quando o projeto chega a um ponto totalmente funcional e pronto para ser entregue:

1. Crie uma tag anotada para registrar a primeira versão estável:
`git tag -a v1.0.0 -m "Versão 1.0.0 pronta para produção"`
2. Para ver as tags existentes no repositório, basta digitar:
`git tag`

Passo 4: O Botão de Pânico (Corrigindo Erros)

Como reverter alterações em diferentes cenários comuns do dia a dia:

Cenário de Erro	Comando de Solução
Cenário A: Alterei o arquivo, mas não adicionei ao stage e me arrependi. Quero voltar ao estado do último commit.	<code>git restore meu_codigo.txt</code>
Cenário B: Adicionei o arquivo ao stage (<code>git add</code>), mas não quero commitá-lo agora.	<code>git restore --staged meu_codigo.txt</code>
Cenário C: Fiz o commit, mas digitei a mensagem errada.	<code>git commit --amend -m "Mensagem corrigida"</code>

3. Atividade Prática: O Laboratório de Design Seguro e Gestão de Crises

A Missão: Você foi contratado(a) como desenvolvedor(a) na startup DevTech. O arquivo `alunos.txt` agora é o "banco de dados" oficial dos colaboradores da empresa. O gerente de projetos pediu para você remodelar o layout visual deste arquivo para torná-lo mais profissional.

Regras de Ouro:

1. Você não pode mexer diretamente no arquivo da linha principal (main ou master).
2. Você recebeu arquivos de configuração pessoal que jamais podem subir para o repositório.

FASE 1: O Escudo de Segurança (Trabalhando com o .gitignore)

1. Abra o terminal (Git Bash) dentro da pasta do seu projeto e crie o `.gitignore`.
2. Abra o arquivo no VS Code e digite a seguinte regra:
`notas_secretas.txt`
3. Crie o arquivo proibido na mesma pasta (`notas_secretas.txt`) e escreva nele uma senha.
4. Rode `git status`. O arquivo não deve aparecer listado, provando que o escudo funciona!
5. Faça o commit do `.gitignore`:

```
git add .gitignore  
git commit -m "Configura regras de segurança no .gitignore"
```

FASE 2: Criando a "Realidade Alternativa" (Branches)

1. Crie e mude para a branch de testes de uma só vez:
`git checkout -b feature/novo-layout`
2. Confirme onde você está listando as branches:
`git branch` (O asterisco deve estar em feature/novo-layout).

FASE 3: O Laboratório de Testes e Simulação de Erros

1. **Cenário A (A Alteração Indesejada):** Abra o alunos.txt, apague tudo e salve. Restaure o arquivo ao estado original usando:
`git restore alunos.txt`
2. **Cenário B (O Falso Positivo no Stage):** Adicione o novo layout e cabeçalhos ao arquivo e rode `git add alunos.txt`. Se arrependeu de ter colocado no stage? Remova com:
`git restore --staged alunos.txt`
3. **Cenário C (Corrigindo a Mensagem do Commit):** Adicione novamente o arquivo e faça o commit com um erro de vocabulário e formatação informal:
`git commit -m "mudei o designer"`
Corrija o termo "designer" (profissional) para "design" (projeto/layout) sem duplicar o commit:
`git commit --amend -m "Aplica layout decorativo ao design do arquivo de alunos"`

FASE 4: O Teste Cósmico e Fusão (Merge e Tag)

1. Volte para a branch principal do projeto:
`git checkout master` (ou `main`).
2. Abra o arquivo alunos.txt e perceba que as alterações sumiram. O projeto principal está seguro.
3. Realize a mesclagem (Fusão) para puxar as modificações validadas da branch de testes:
`git merge feature/novo-layout`
4. Delete a branch de teste para manter a organização:
`git branch -d feature/novo-layout`
5. Crie a Tag de Lançamento para marcar a nova versão visual:
`git tag -a v1.1.0 -m "Lançamento oficial do novo layout dos colaboradores"`
6. Verifique seu histórico final com `git log --oneline --graph` para ver a tag brilhar ao lado do commit!

Critérios de Sucesso (Checklist)

- O arquivo .gitignore existe e impede o rastreamento de notas_secretas.txt.
- A branch feature/novo-layout foi criada, mesclada sem conflitos e posteriormente deletada.
- O aluno utilizou comandos de correção (`git restore` ou `git commit --amend`) com sucesso.
- A tag v1.1.0 foi criada no último commit da linha principal.